Vare AI — Master Backend Specification

For Replit Agent Implementation

Version 1.0 — March 2026

---

1. Project Overview

Build a Python backend that powers the Vare AI platform. The backend handles three core functions:

1. **Merchant Onboarding** — A multi-phase wizard that connects a Magento merchant's catalog, ingests products, normalizes data, configures inventory probing, and sets up agent-facing endpoints.
2. **Dashboard API** — REST endpoints serving metrics, analytics, and pipeline health data to the Vare AI Pulse Dashboard frontend (already built in Lovable).
3. **Agent-Facing API** — MCP-compatible endpoints that AI agents query to discover and purchase products.

The frontend is a separate React app hosted on Lovable. This backend is a standalone API service.

---

2. Tech Stack

- **Language:** Python 3.11+
- **Framework:** FastAPI (async, automatic OpenAPI docs, easy to deploy)
- **Database:** PostgreSQL (use Replit's built-in PostgreSQL or connect to an external instance)
- **ORM:** SQLAlchemy with async support

- **Task Queue:** For long-running operations (catalog sync, normalization), use Python asyncio with background tasks initially. Migrate to Celery + Redis later if needed.
- **LLM Integration:** Anthropic Claude API for normalization tasks (fuzzy matching, description generation, attribute enrichment)
- **HTTP Client:** httpx (async) for Magento API calls
- **Auth:** Bearer token authentication (simple API key per merchant workspace)
- **Deployment:** Replit (initially), designed for easy migration to Railway/Render/AWS later

---

3. Database Schema

3.1 Core Tables

```sql
```

```sql
-- Merchant accounts
CREATE TABLE merchants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_name VARCHAR(200) NOT NULL,
    contact_first_name VARCHAR(100),
    contact_last_name VARCHAR(100),
    contact_email VARCHAR(255) NOT NULL,
    contact_phone VARCHAR(50),
    estimated_sku_count VARCHAR(50), -- "Under 1,000", "1,000-10,000", etc
    primary_vertical VARCHAR(100), -- "Automotive Parts & Accessories", et
    magento_version VARCHAR(50), -- "magento2_commerce", "magento2_opensou
    hosting_environment VARCHAR(100),
    erp_system VARCHAR(100),
    pim_system VARCHAR(100),
    complexity_score INTEGER DEFAULT 0, -- 0-4, computed from profile
    onboarding_phase INTEGER DEFAULT 1, -- current wizard phase (1-10)
    onboarding_status VARCHAR(20) DEFAULT 'in_progress', -- in_progress, c
    api_key VARCHAR(255), -- merchant's Vare AI API key
    sandbox_mode BOOLEAN DEFAULT true,
    is_live BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Magento connection credentials (encrypted at rest)
CREATE TABLE magento_connections (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    store_url VARCHAR(500) NOT NULL,
    consumer_key VARCHAR(500),
    consumer_secret VARCHAR(500),
    access_token VARCHAR(500),
    access_token_secret VARCHAR(500),
    -- For Magento 1.x
    api_user VARCHAR(255),
    api_key_m1 VARCHAR(500),
```

```sql
    -- Connection metadata
    store_name VARCHAR(255),
    detected_version VARCHAR(100),
    base_currency VARCHAR(10),
    locale VARCHAR(20),
    connection_status VARCHAR(20) DEFAULT 'pending', -- pending, connecte
    last_health_check TIMESTAMP,
    api_health_pct FLOAT DEFAULT 100.0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Store views selected for sync
CREATE TABLE store_views (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    magento_store_view_id INTEGER,
    code VARCHAR(100),
    name VARCHAR(255),
    is_selected BOOLEAN DEFAULT true,
    is_default BOOLEAN DEFAULT false
);

-- Raw product data from Magento (staging)
CREATE TABLE raw_products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    magento_product_id INTEGER,
    sku VARCHAR(255) NOT NULL,
    product_type VARCHAR(50), -- simple, configurable, grouped, bundle
    status VARCHAR(20), -- enabled, disabled
    visibility VARCHAR(50),
    raw_data JSONB NOT NULL, -- full Magento API response
    synced_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_raw_products_merchant_sku ON raw_products(merchant_id, s
```

```sql
-- Normalized product data (agent-ready)
CREATE TABLE normalized_products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    raw_product_id UUID REFERENCES raw_products(id),
    sku VARCHAR(255) NOT NULL,
    product_title VARCHAR(500),
    description TEXT,
    short_description TEXT,
    brand VARCHAR(255),
    manufacturer VARCHAR(255),
    mpn VARCHAR(255), -- manufacturer part number
    upc VARCHAR(50),
    price DECIMAL(10,2),
    currency VARCHAR(10) DEFAULT 'USD',
    color VARCHAR(100),
    finish VARCHAR(100),
    weight DECIMAL(10,4),
    weight_unit VARCHAR(10),
    category_path VARCHAR(500), -- "Auto Parts > Electrical > Alternators"
    image_urls JSONB, -- array of image URLs
    custom_attributes JSONB, -- all other normalized attributes
    fitment_data JSONB, -- structured fitment: [{year, make, model, engine
    agent_readiness_score INTEGER, -- 0-100 per product
    normalization_status VARCHAR(20) DEFAULT 'pending', -- pending, normal
    normalized_at TIMESTAMP,
    updated_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_normalized_products_merchant ON normalized_products(merc
CREATE INDEX idx_normalized_products_sku ON normalized_products(merchant_

-- Attribute mappings (merchant attribute -> Vare universal attribute)
CREATE TABLE attribute_mappings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
```

```sql
    source_attribute VARCHAR(255) NOT NULL, -- Magento attribute code
    target_attribute VARCHAR(255), -- Vare universal attribute name
    mapping_status VARCHAR(20) DEFAULT 'auto', -- auto, reviewed, manual,
    confidence FLOAT, -- 0-1, model confidence in auto-mapping
    data_type VARCHAR(50), -- measurement, category, boolean, identifier,
    normalization_unit VARCHAR(50), -- for measurements: "inches", "mm", 
    created_at TIMESTAMP DEFAULT NOW()
);

-- Value normalization rules (e.g., "Blk" -> "Black")
CREATE TABLE value_normalizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    attribute_mapping_id UUID REFERENCES attribute_mappings(id),
    source_value VARCHAR(500) NOT NULL,
    normalized_value VARCHAR(500) NOT NULL,
    cluster_name VARCHAR(255), -- grouping label, e.g., "Black"
    status VARCHAR(20) DEFAULT 'suggested', -- suggested, accepted, rejec
    product_count INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Inventory state
CREATE TABLE inventory (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    sku VARCHAR(255) NOT NULL,
    quantity INTEGER,
    is_in_stock BOOLEAN,
    low_stock_threshold INTEGER DEFAULT 5,
    source_name VARCHAR(255), -- warehouse name
    last_probed TIMESTAMP,
    probe_latency_ms INTEGER,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE INDEX idx_inventory_merchant_sku ON inventory(merchant_id, sku);

-- Inventory probe configuration
CREATE TABLE probe_configs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    inventory_source VARCHAR(50) DEFAULT 'magento', -- magento, erp, custo
    probe_frequency VARCHAR(50) DEFAULT 'cached', -- continuous, cached, b
    cache_ttl_minutes INTEGER DEFAULT 5,
    fallback_behavior VARCHAR(50) DEFAULT 'last_known', -- out_of_stock, l
    low_stock_threshold INTEGER DEFAULT 5,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Sync jobs (catalog sync, normalization runs, etc.)
CREATE TABLE sync_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    job_type VARCHAR(50) NOT NULL, -- catalog_full, catalog_delta, normali
    status VARCHAR(20) DEFAULT 'queued', -- queued, running, completed, fa
    total_records INTEGER DEFAULT 0,
    processed_records INTEGER DEFAULT 0,
    error_count INTEGER DEFAULT 0,
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    duration_seconds INTEGER,
    error_log JSONB, -- array of {sku, error, timestamp}
    config JSONB, -- job-specific config (product types, filters, etc.)
    created_at TIMESTAMP DEFAULT NOW()
);

-- Orders placed via AI agents
CREATE TABLE agent_orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    magento_order_id VARCHAR(100),
```

```sql
    agent_platform VARCHAR(50), -- chatgpt, gemini, claude, perplexity, m
    agent_session_id VARCHAR(255),
    sku VARCHAR(255),
    product_title VARCHAR(500),
    quantity INTEGER,
    unit_price DECIMAL(10,2),
    total_price DECIMAL(10,2),
    order_status VARCHAR(50), -- pending, processing, shipped, delivered,
    payment_method VARCHAR(100),
    shipping_method VARCHAR(100),
    error_message TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Agent queries log (for analytics)
CREATE TABLE agent_queries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    agent_platform VARCHAR(50),
    query_text TEXT,
    matched_skus JSONB, -- array of SKUs returned
    result_count INTEGER DEFAULT 0,
    was_matched BOOLEAN DEFAULT false,
    intent_cluster VARCHAR(255), -- clustered intent, e.g., "Brake pads f
    session_id VARCHAR(255),
    response_time_ms INTEGER,
    created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_agent_queries_merchant_date ON agent_queries(merchant_id

-- Transaction events (full funnel tracking)
CREATE TABLE transaction_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    session_id VARCHAR(255),
```

```sql
    agent_platform VARCHAR(50),
    sku VARCHAR(255),
    event_type VARCHAR(50), -- query, product_returned, detail_viewed, ca
    status VARCHAR(20) DEFAULT 'success', -- success, failed, pending
    duration_ms INTEGER,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_transaction_events_merchant_date ON transaction_events(m

-- System alerts
CREATE TABLE system_alerts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    alert_type VARCHAR(20), -- warning, success, info, error
    title VARCHAR(255),
    description TEXT,
    suggestion TEXT,
    is_read BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Insights (AI-generated, cached)
CREATE TABLE insights (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    insight_type VARCHAR(50), -- trending_up, lost_revenue, top_performer
    badge VARCHAR(100),
    text TEXT,
    action_label VARCHAR(100),
    date_range VARCHAR(20),
    created_at TIMESTAMP DEFAULT NOW()
);
```

4. API Endpoints

4.1 Authentication

All endpoints require a Bearer token in the Authorization header. Tokens are merchant-scoped (one token per merchant workspace).

```
Authorization: Bearer vare_live_sk_xxxxxxxxxxxxx
```

Middleware extracts merchant_id from the token and scopes all queries accordingly.

4.2 Onboarding Endpoints

```
POST   /api/onboarding/merchant              — Create merchant profile
(Phase 1)
PATCH  /api/onboarding/merchant/{id}          — Update merchant profile
GET    /api/onboarding/merchant/{id}          — Get current onboarding
state
GET    /api/onboarding/merchant/{id}/complexity — Get computed
complexity score

POST   /api/onboarding/connect                — Submit Magento
credentials (Phase 2)
POST   /api/onboarding/connect/test           — Test Magento API
connection
GET    /api/onboarding/connect/health          — Run deep connection
health check
GET    /api/onboarding/connect/store-views     — List detected store
views
PATCH  /api/onboarding/connect/store-views     — Update store view
selections

POST   /api/onboarding/sync/configure          — Set sync filters
(Phase 3)
POST   /api/onboarding/sync/start              — Start catalog sync
GET    /api/onboarding/sync/status             — Get sync progress
```

```
POST   /api/onboarding/sync/pause              — Pause running sync
POST   /api/onboarding/sync/cancel             — Cancel running sync
GET    /api/onboarding/sync/summary            — Get sync completion
summary
GET    /api/onboarding/sync/errors             — Get sync error log

GET    /api/onboarding/health-scan             — Get data health scan
results (Phase 4)
GET    /api/onboarding/health-scan/attributes  — Get attribute coverage
heatmap
GET    /api/onboarding/health-scan/issues      — Get issue breakdown
cards

GET    /api/onboarding/normalization/attributes — List attribute
mappings (Phase 5)
PATCH  /api/onboarding/normalization/attributes/{id} — Update a mapping
GET    /api/onboarding/normalization/values/{attribute_id} — Get value
clusters for an attribute
PATCH  /api/onboarding/normalization/values/{id} — Accept/reject/edit a
value normalization
POST   /api/onboarding/normalization/preview   — Generate normalization
preview (sample products)
POST   /api/onboarding/normalization/apply      — Apply normalization
to all products

GET    /api/onboarding/fitment/assessment       — Get fitment data
assessment (Phase 6)
POST   /api/onboarding/fitment/sources          — Configure fitment
data sources
GET    /api/onboarding/fitment/preview          — Preview fitment
matches with confidence
POST   /api/onboarding/fitment/apply            — Apply fitment
enrichment

POST   /api/onboarding/probe/configure          — Set probe config
(Phase 7)
```

```
POST   /api/onboarding/probe/test              — Run probe test on
sample products
GET    /api/onboarding/probe/results            — Get probe test
results

POST   /api/onboarding/gateway/configure        — Set order gateway
config (Phase 8)
POST   /api/onboarding/gateway/test-order        — Place a test order
DELETE /api/onboarding/gateway/test-order/{id}   — Cancel test order

GET    /api/onboarding/agent-config              — Get agent visibility
settings (Phase 9)
PATCH  /api/onboarding/agent-config              — Update agent
visibility settings
POST   /api/onboarding/agent-config/generate-key — Generate API key

GET    /api/onboarding/review                    — Get full review
summary (Phase 10)
POST   /api/onboarding/activate                  — Go live (or activate
sandbox)
```

4.3 Dashboard Metrics Endpoints (Agent View)

All accept `?range=today|7d|30d|90d|ytd` query parameter.

```
GET  /api/metrics/kpi?metric={key}            — Single KPI card data
     Keys: revenue, orders, queries, conversion, aov, platforms
     Response: { key, title, value, change, changeType, sparklineData[]
}

GET  /api/metrics/timeseries?metric={key}&range={range}
     Response: { dates[], values[] }

GET  /api/metrics/platform-breakdown?range={range}
     Response: [{ name, value, percentage }]

GET  /api/metrics/top-products?range={range}&limit=50&offset=0
     Response: { data: [{ rank, product, sku, revenue, orders, queries,
convRate, trend }], total }

GET  /api/metrics/query-intents?range={range}
     Response: [{ intent, queries, matched, convRate, example }]

GET  /api/metrics/unmatched-queries?range={range}&limit=50
     Response: { data: [{ query, count, reason }], summary: {
total_queries, matched_pct, unmatched_count, unmatched_pct } }

GET  /api/metrics/conversion-funnel?range={range}
     Response: { steps: [{ label, value, percentage, dropRate }],
drop_off_insights[] }

GET  /api/metrics/failed-transactions?range={range}&limit=50
     Response: { data: [{ date, agent, product, error, resolution }],
summary: { total_failed, success_rate, top_error, top_error_pct },
failure_breakdown[] }

GET  /api/insights?range={range}
     Response: [{ type, badge, date, text, action }]
```

4.4 Dashboard Metrics Endpoints (Feeds View)

```
GET  /api/feeds/connections
     Response: [{ name, type, status, lastSync, metric, metricValue,
apiHealth }]

GET  /api/feeds/sync-timeline
     Response: [{ label, segments: [{ type, width, detail }] }]

GET  /api/feeds/readiness-score
     Response: { currentScore, targetScore, trendData[], breakdown[],
opportunity }

GET  /api/feeds/sync-history?page=1&limit=25
     Response: { data: [{ date, source, type, records, duration,
status, details }], total }
GET  /api/feeds/sync-history/export?format=csv

GET  /api/feeds/data-quality
     Response: { attributes: [{ name, withData, without, coverage,
trend30d, agentCritical }], coverageOverTime[] }

GET  /api/feeds/normalization
     Response: { stats: { totalAttributes, activeMappings,
autoNormalized, pendingReview }, recentActivity[] }

GET  /api/feeds/errors?range={range}
     Response: { summary: { total, errors, warnings, trend },
categories[] }
GET  /api/feeds/errors/export?format=csv

GET  /api/feeds/inventory-health
     Response: { totalSKUs, inStock, lowStock, outOfStock,
dataFreshness, accuracy, topStockOuts[] }

GET  /api/feeds/system-alerts
```

```
    Response: [{ type, title, time, description, suggestion, actions[]
}]
```

4.5 Transaction Ledger

```
GET  /api/transactions?page=1&limit=50&search=&filter=
    Response: { data: [{ id, timestamp, agent, sku, action, status,
duration, details[] }], total }

GET  /api/bot-defense/settings
    Response: [{ agentId, name, status, requestsToday, lastActive }]
PATCH /api/bot-defense/settings/{agentId}
    Body: { status: "enabled" | "disabled" }
```

4.6 Agent-Facing API (MCP-compatible)

These endpoints are what AI agents call to discover and purchase products.

```
GET  /api/v1/merchants/{merchant_slug}/catalog
     Query params: q (search query), category, brand, min_price,
max_price,
                   fitment_year, fitment_make, fitment_model,
fitment_engine,
                   page, limit
     Response: { products: [normalized product objects], total, facets
}

GET  /api/v1/merchants/{merchant_slug}/catalog/{sku}
     Response: full normalized product with fitment, images, inventory
status

GET  /api/v1/merchants/{merchant_slug}/inventory/{sku}
     Response: { sku, in_stock, quantity, low_stock, sources[],
last_checked }
     This triggers the Synthetic Probe if cache is stale.

POST /api/v1/merchants/{merchant_slug}/cart
     Body: { items: [{ sku, quantity }], shipping_address }
     Response: { cart_id, items, subtotal, tax, shipping, total }

POST /api/v1/merchants/{merchant_slug}/checkout
     Body: { cart_id, payment_token, agent_id }
     Response: { order_id, status, confirmation }
     This injects the order into Magento (or ERP).
```

---

5. Core Services (Business Logic)

5.1 Magento Connector Service

Handles all communication with the Magento REST API (v2) and SOAP API (v1).

```python
```

```python
class MagentoConnector:
    """
    Responsibilities:
    - Test connection with provided credentials
    - Fetch store configuration and store views
    - Fetch products in batches (paginated, with filters)
    - Fetch categories tree
    - Fetch product attributes list
    - Check inventory (single SKU and batch)
    - Create orders
    - Handle Magento 1.x SOAP separately
    """

    async def test_connection(self, credentials) -> ConnectionResult:
        # GET {store_url}/rest/V1/store/storeConfigs
        # Timeout: 15 seconds
        # Parse: store name, version, currency, store views, locale
        # Return structured result with success/failure and error details

    async def fetch_products(self, filters, page, page_size=100) -> Produ
        # GET {store_url}/rest/V1/products?searchCriteria[...]
        # Apply filters: status, visibility, product_type, category, updat
        # For configurable products, also fetch child simple products
        # Return raw product data as JSONB

    async def fetch_categories(self) -> CategoryTree:
        # GET {store_url}/rest/V1/categories

    async def fetch_attributes(self) -> list[Attribute]:
        # GET {store_url}/rest/V1/products/attributes?searchCriteria[page

    async def check_inventory(self, sku) -> InventoryResult:
        # GET {store_url}/rest/V1/stockStatuses/{sku}
        # Or for MSI: GET {store_url}/rest/V1/inventory/source-items?...
        # Record latency
```

```
    async def create_order(self, order_data) -> OrderResult:
        # POST {store_url}/rest/V1/carts/mine (create cart)
        # POST {store_url}/rest/V1/carts/mine/items (add items)
        # POST {store_url}/rest/V1/carts/mine/shipping-information
        # POST {store_url}/rest/V1/carts/mine/payment-information (place o

    async def health_check(self) -> HealthCheckResult:
        # Test: API response time, catalog endpoint, inventory endpoint,
        #        order creation permissions (dry run), image CDN reachabi
```

5.2 Catalog Sync Service

Orchestrates the full catalog pull from Magento into the raw_products table.

```python
class CatalogSyncService:
    """
    Responsibilities:
    - Create and manage sync jobs
    - Pull products in batches, respecting sync configuration
    - Track progress and errors in real time
    - Support pause/resume/cancel
    - Delta syncs using updated_at filter
    """

    async def start_full_sync(self, merchant_id, config) -> SyncJob:
        # Create sync_job record
        # Fetch total product count matching filters
        # Loop: fetch batch -> store in raw_products -> update progress
        # Handle errors per-product (retry 3x, then skip and log)
        # On completion: compute summary stats

    async def start_delta_sync(self, merchant_id) -> SyncJob:
        # Same as full sync but filter by updated_at > last_sync_timestamp
        # Only update changed products
```

5.3 Normalization Service

The core intelligence of Vare AI. Transforms raw product data into agent-ready structured data.

```python
```

```python
class NormalizationService:
    """
    Three-layer normalization pipeline:

    Layer 1: Rules-based (fast, deterministic, free)
    - Lookup tables for known value mappings (loaded per vertical)
    - Regex patterns for unit extraction and standardization
    - Direct field mapping (name -> product_title, etc.)

    Layer 2: LLM-assisted (for ambiguous cases)
    - Send product record + vertical context to Claude API
    - Ask for: missing attribute inference, jargon translation,
      description generation, category suggestion
    - Parse structured response

    Layer 3: Human review queue (for low-confidence results)
    - Flag products where confidence < threshold
    - Present to merchant in dashboard for approval
    """

    async def auto_map_attributes(self, merchant_id) -> list[AttributeMap
        # Analyze raw_products attribute names and values
        # Match against Vare universal schema using string similarity + L
        # Return mappings with confidence scores

    async def cluster_values(self, merchant_id, attribute) -> list[ValueC
        # Group raw values into semantic clusters
        # "Blk", "BLACK", "Black", "Mid_Blk" -> cluster "Black"
        # Use string similarity + LLM for ambiguous cases
        # Detect sub-variants (Matte, Gloss, etc.)

    async def normalize_product(self, raw_product, mappings, rules) -> No
        # Apply attribute mappings
        # Apply value normalization rules
        # For missing critical attributes: call LLM for inference
        # Compute per-product agent-readiness score
```

```
        # Return normalized product

    async def normalize_batch(self, merchant_id) -> NormalizationJob:
        # Process all raw_products for merchant
        # Apply rules first (fast pass)
        # Batch LLM calls for remaining gaps
        # Store results in normalized_products
        # Track progress

    async def compute_readiness_score(self, merchant_id) -> ReadinessScor
        # Weighted score across:
        #   Attribute Completeness (30%)
        #   Title Quality (15%)
        #   Description Quality (10%)
        #   Image Coverage (10%)
        #   Price Accuracy (10%)
        #   Category Depth (10%)
        #   Fitment Data (15%) -- only for automotive/industrial
```

5.4 Fitment Service (Automotive/Industrial)

```python
```

```python
class FitmentService:
    """
    Handles structured Year/Make/Model/Engine fitment data.
    Only active for automotive and industrial verticals.
    """

    async def assess_fitment(self, merchant_id) -> FitmentAssessment:
        # Count: products with structured fitment, text-based fitment, no

    async def enrich_from_aces(self, aces_data, merchant_id):
        # Parse ACES XML
        # Match SKUs to fitment records
        # Store in normalized_products.fitment_data

    async def extract_from_descriptions(self, merchant_id):
        # Use LLM to extract Year/Make/Model/Engine from description text
        # Flag confidence levels
        # Store as low-confidence fitment data for merchant review

    async def cross_reference_mpn(self, merchant_id):
        # Use manufacturer part numbers to look up fitment from reference
```

5.5 Inventory Probe Service (Synthetic Probe)

```python
```

```python
class InventoryProbeService:
    """
    Real-time inventory verification.
    """

    async def probe_single(self, merchant_id, sku) -> ProbeResult:
        # Check cache first (respect TTL from probe_config)
        # If stale: call Magento inventory API
        # Record latency
        # Update inventory table
        # Return result with freshness indicator

    async def probe_batch(self, merchant_id, skus) -> list[ProbeResult]:
        # Batch inventory check for multiple SKUs
        # Used for scheduled batch probes

    async def handle_fallback(self, merchant_id, sku) -> ProbeResult:
        # If probe fails: apply fallback behavior from config
        # out_of_stock: report unavailable
        # last_known: return cached data with stale flag
        # in_stock: report available (risky)
```

5.6 Order Injection Service

```python
```

```python
class OrderInjectionService:
    """
    Creates orders in the merchant's Magento instance when an agent compl
    """

    async def create_order(self, merchant_id, order_data) -> OrderResult:
        # Verify inventory one more time (probe)
        # Create cart in Magento
        # Add items
        # Set shipping
        # Apply payment (Vare AI payment method)
        # Place order
        # Tag with order_source = "vare_ai" and agent_id
        # Log in agent_orders table
        # Return confirmation

    async def create_test_order(self, merchant_id) -> OrderResult:
        # Same flow but with $0.00 test product
        # Flag as test in metadata

    async def cancel_order(self, merchant_id, order_id) -> bool:
        # Cancel via Magento API
```

5.7 Metrics Aggregation Service

```python
```

```
class MetricsService:
    """
    Computes and caches dashboard metrics from raw event data.
    """

    async def get_kpi(self, merchant_id, metric, date_range) -> KPIResult
        # Query agent_orders and agent_queries tables
        # Compute: revenue, orders, queries, conversion, aov, active plat
        # Include change vs prior period
        # Generate sparkline data (15 points)

    async def get_timeseries(self, merchant_id, metric, date_range) -> Ti
        # Daily aggregation of the selected metric

    async def get_platform_breakdown(self, merchant_id, date_range) -> li
        # Group agent_orders by agent_platform

    async def get_conversion_funnel(self, merchant_id, date_range) -> Fun
        # Count transaction_events at each stage
        # Compute drop-off rates between stages

    async def generate_insights(self, merchant_id, date_range) -> list[In
        # Analyze recent data for patterns
        # Use LLM to generate natural language insights
        # Cache results
```

---

6. Background Jobs

These run on a schedule or are triggered by events:

Job	Trigger	Description
Delta catalog sync	Every 6 hours (configurable)	Pull changed products from Magento
Inventory batch probe	Per probe_config schedule	Refresh inventory for all active SKUs
Readiness score recompute	After any normalization run	Recompute agent-readiness scores
Insight generation	Daily	Generate AI insights from recent metrics
Health check	Every 5 minutes	Verify Magento API connectivity
Alert generation	On health check failure	Create system_alert if connection degrades
Sync timeline update	Continuous	Update sync activity timeline data

7. API Response Standards

All endpoints follow these patterns:

```
json
```

```
// Success (single object)
{ "data": { ... }, "generated_at": "2026-03-20T12:00:00Z" }

// Success (paginated list)
{ "data": [ ... ], "total": 1000, "page": 1, "limit": 50, "generated_at":

// Error
{ "error": "Description of error", "code": "INVALID_CREDENTIALS", "detail
```

All monetary values in USD (or merchant's configured currency). All timestamps in ISO 8601. Sparkline arrays contain exactly 15 data points. Paginated endpoints accept `page` and `limit` params.

---

8. Environment Variables

```
DATABASE_URL=postgresql://...
ANTHROPIC_API_KEY=sk-ant-...
VARE_API_SECRET=... (for generating merchant API keys)
ENCRYPTION_KEY=... (for encrypting Magento credentials at rest)
```

---

9. File Structure

```
vare-ai-backend/
├── main.py                    # FastAPI app entry point
├── config.py                  # Environment vars, settings
├── database.py                # DB connection, session management
├── models/                    # SQLAlchemy models
│   ├── merchant.py
│   ├── connection.py
│   ├── product.py
│   ├── normalization.py
```

```
│   ├── inventory.py
│   ├── order.py
│   ├── analytics.py
│   └── alert.py
├── routes/                     # API route handlers
│   ├── onboarding.py
│   ├── metrics.py
│   ├── feeds.py
│   ├── transactions.py
│   ├── agent_api.py            # MCP-compatible agent-facing endpoints
│   └── bot_defense.py
├── services/                   # Business logic
│   ├── magento_connector.py
│   ├── catalog_sync.py
│   ├── normalization.py
│   ├── fitment.py
│   ├── inventory_probe.py
│   ├── order_injection.py
│   ├── metrics_aggregation.py
│   └── insights.py
├── tasks/                      # Background jobs
│   ├── scheduled_sync.py
│   ├── inventory_refresh.py
│   ├── health_monitor.py
│   └── insight_generator.py
├── utils/
│   ├── auth.py                 # Token validation middleware
│   ├── encryption.py           # Credential encryption
│   ├── magento_errors.py       # Error mapping for Magento API
responses
│   └── normalization_rules/    # Per-vertical rules
│       ├── automotive.py
│       ├── industrial.py
│       ├── electronics.py
│       └── base.py
├── data/                       # Reference data files
```

```
│   ├── color_mappings.json
│   ├── unit_conversions.json
│   └── vertical_schemas/
│       ├── automotive.json    # Agent-critical attributes for auto
│       ├── industrial.json
│       └── electronics.json
├── requirements.txt
└── README.md
```

---

10. Build Priority Order

Build in this order. Each phase is testable independently.

Phase 1: Foundation (Week 1)

- FastAPI app skeleton with health endpoint
- Database connection and table creation
- Merchant CRUD (create, read, update)
- Authentication middleware
- Basic error handling

Phase 2: Magento Connection (Week 1-2)

- MagentoConnector service: test_connection, fetch_store_config
- Onboarding connect endpoints
- Health check endpoint
- Store view detection
- Handle all Magento error codes (401, 403, 404, 500, timeout, SSL)

Phase 3: Catalog Sync (Week 2-3)

- Product batch fetching with pagination
- Category tree fetching

- Attribute list fetching
- Sync job tracking (progress, errors)
- Raw product storage
- Delta sync capability (updated_at filter)
- Sync configuration (product types, status, visibility, categories, attributes)

Phase 4: Data Health Scan (Week 3)

- Attribute coverage analysis
- Agent-readiness score computation (weighted formula)
- Issue detection (missing descriptions, thin titles, no images, etc.)
- Health scan endpoints

Phase 5: Normalization Engine (Week 3-4)

- Attribute auto-mapping (string similarity + LLM)
- Value clustering (group similar values)
- Rules-based normalization (lookup tables, regex, unit conversion)
- LLM-based normalization (Claude API calls for ambiguous cases)
- Normalization preview (sample products)
- Batch normalization with progress tracking
- Per-vertical rule sets (start with automotive)

Phase 6: Inventory Probe (Week 4)

- Single SKU probe via Magento API
- Cache layer with configurable TTL
- Batch probe
- Fallback behavior
- Latency tracking

Phase 7: Order Injection (Week 4-5)

- Cart creation via Magento API
- Item addition
- Shipping and payment configuration
- Order placement
- Test order flow
- Order tagging (vare_ai source)

Phase 8: Agent-Facing API (Week 5)

- Product search endpoint (query, filters, facets)
- Single product detail endpoint
- Inventory check endpoint (triggers probe)
- Cart and checkout endpoints
- Request logging to agent_queries table

Phase 9: Dashboard Metrics (Week 5-6)

- KPI computation from agent_queries and agent_orders
- Time series aggregation
- Platform breakdown
- Conversion funnel from transaction_events
- Top products ranking
- All /api/metrics/* and /api/feeds/* endpoints
- Mock data seeding for demo purposes until real traffic exists

Phase 10: Background Jobs (Week 6)

- Scheduled delta sync
- Inventory refresh
- Health monitoring

- Insight generation

---

11. Testing Strategy

For each phase, create test endpoints that can be called manually:

```
POST /api/test/seed-mock-data          — Populate tables with realistic
sample data
POST /api/test/simulate-agent-query    — Simulate an agent searching
the catalog
POST /api/test/simulate-order          — Simulate an agent placing an
order
GET  /api/test/health                  — Full system health check
```

Also seed a demo Magento instance for end-to-end testing. Magento 2 Open Source can be run via Docker (use the official `magento/magento-cloud-docker` images) or use a free trial of Adobe Commerce Cloud.

---

12. CORS Configuration

The frontend is hosted on Lovable (vare-ai.com). Configure CORS to allow:

```python
origins = [
    "https://vare-ai.com",
    "http://localhost:3000",    # local frontend dev
    "http://localhost:5173",    # Vite dev server
]
```

---

13. Initial Mock Data

Until real merchants are connected, the dashboard needs realistic mock data. Build a seeder that creates:

- 1 merchant ("Acme Auto Parts")
- 50,000 mock products with varying completeness levels
- 30 days of simulated agent queries (1,000/day with realistic distribution)
- 30 days of simulated orders (20-30/day)
- Transaction events for the full funnel
- Normalization history
- Sync history records
- System alerts (mix of warnings, successes, info)
- Insights

This mock data should make the dashboard fully functional for demos even without a real Magento connection.